Avaliação Experimental do Bubble Sort Clássico e Otimizado em Comparação com o Merge Sort

Matheus Castilho

Universidade Tecnológica Federal do Paraná (UTFPR)
Medianeira – PR – Brasil

Abstract. *This paper presents an experimental evaluation comparing the classic and optimized (early stop flag) versions of the Bubble Sort algorithm with Merge Sort, focusing on reading and processing datasets of different scales. The objective is to analyze the impact of input data scale and initial ordering on execution time and algorithm efficiency, investigating to what extent the early stopping mechanism mitigates the quadratic complexity of Bubble Sort compared to the* $\Theta(n \log n)$ *performance of Merge Sort. Empirical results highlight the practical limits of this optimization under best, average, and worst-case scenarios, demonstrating the performance of each approach in high-demand workloads.*

Resumo. *Este trabalho apresenta uma avaliação experimental comparativa entre os algoritmos de ordenação Bubble Sort (em suas versões clássica e otimizada por parada antecipada) e Merge Sort, focando na leitura e no processamento de diferentes escalas de dados. O objetivo é analisar o impacto do tamanho e da ordenação inicial dos dados no tempo de execução e na eficiência dos algoritmos, investigando até que ponto o mecanismo de interrupção precoce atenua a complexidade quadrática do Bubble Sort frente ao desempenho* $\Theta(n \log n)$ *do Merge Sort. Os resultados empíricos destacam os limites práticos dessa otimização em cenários de melhor, médio e pior caso, evidenciando o comportamento de cada abordagem sob cargas de trabalho de alta demanda.*

1. Introdução

A ordenação de dados é uma operação fundamental na Ciência da Computação, sendo essencial para otimizar operações de busca e o processamento eficiente de informações (CORMEN et al., 2012). Com o crescimento contínuo do volume de dados em aplicações reais, a eficiência assintótica e o desempenho prático dos algoritmos de ordenação constituem fatores determinantes para o rendimento global dos sistemas (KNUTH, 1998).

Algoritmos iterativos simples, como o *Bubble Sort* clássico, possuem complexidade temporal $\Theta(n^2)$ em todos os cenários (melhor, médio e pior caso), pois realizam $n - 1$ passagens pelo vetor e $n(n-1)/2$ comparações entre elementos adjacentes, independentemente do arranjo inicial dos dados (CORMEN et al., 2012). A inclusão de uma otimização com sinalizador (flag) de parada antecipada permite interromper a execução quando nenhuma troca é efetuada em uma varredura completa, reduzindo a complexidade no melhor caso (dados pré-ordenados) para $\Theta(n)$ (ASTRACHAN, 2003; SEDGEWICK; WAYNE, 2011). Em contrapartida, algoritmos baseados no paradigma de divisão e conquista, como o *Merge Sort*, oferecem desempenho temporal assintótico $\Theta(n \log n)$ em qualquer configuração inicial dos dados, embora exijam memória auxiliar $O(n)$ durante a intercalação (CORMEN et al., 2012).

Este trabalho apresenta uma avaliação experimental comparativa entre o *Bubble Sort* clássico, o *Bubble Sort* otimizado (com critério de parada antecipada) e o *Merge Sort*, com o objetivo de avaliar o impacto da interrupção precoce no tempo de execução e comparar o desempenho dos algoritmos diante de diferentes tamanhos e graus de ordenação inicial da entrada.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 discute os trabalhos relacionados; a Seção 4 descreve a metodologia experimental; a Seção 5 apresenta e analisa os resultados; e a Seção 6 reúne as considerações finais.

2. Fundamentação Teórica

2.1 Bubble Sort

O *Bubble Sort* é um dos algoritmos de ordenação por comparação mais conhecidos e intuitivos na Ciência da Computação (CORMEN et al., 2012; KNUTH, 1998). Sua estratégia fundamenta-se em percorrer iterativamente o vetor, comparando pares de elementos adjacentes e efetuando a troca de suas posições sempre que o elemento localizado à esquerda for estritamente maior que o da direita (SEDGEWICK; WAYNE, 2011). A cada passagem completa do laço externo, o maior elemento remanescente da porção não ordenada "flutua" progressivamente até sua posição definitiva na extremidade direita do vetor, reduzindo a dimensão da varredura na iteração subsequente (LEVITIN, 2012).

Em sua formulação clássica, a complexidade temporal do *Bubble Sort* é de $\Theta(n^2)$ nos cenários de melhor, médio e pior caso (CORMEN et al., 2012). Isso ocorre porque o algoritmo executa invariavelmente $n - 1$ passagens pelo laço externo e realiza exatamente $n(n - 1)/2$ comparações, independentemente do arranjo inicial dos dados de entrada (KNUTH, 1998; ASTRACHAN, 2003).

Quanto ao uso de memória, sua complexidade espacial é de $O(1)$, operando de maneira *in-place* por exigir apenas uma quantidade constante de memória auxiliar (SEDGEWICK; WAYNE, 2011). Entre suas principais qualidades sobressaem-se a simplicidade de implementação e a estabilidade nativa, mantendo a ordem relativa de elementos com chaves idênticas (GOODRICH; TAMASSIA, 2013). Em contrapartida, sua desvantagem central consiste na acentuada ineficiência assintótica em grandes volumes de dados, agravada pelo elevado custo computacional decorrente das sucessivas trocas de elementos na memória (ASTRACHAN, 2003; SKIENA, 2020).

2.2 Bubble Sort Otimizado (Parada Antecipada)

O *Bubble Sort* Otimizado consiste em uma variação projetada para mitigar o tempo de processamento quando aplicado a conjuntos de dados total ou parcialmente ordenados (ASTRACHAN, 2003). A modificação estrutural baseia-se na introdução de um sinalizador booleano (*flag*) de controle, responsável por registrar a ocorrência de trocas durante a execução do laço interno (SEDGEWICK; WAYNE, 2011; LEVITIN, 2012). Caso uma varredura completa seja concluída sem a realização de nenhuma troca, o algoritmo detecta antecipadamente que a sequência já se encontra totalmente ordenada e encerra a execução do laço externo de forma prematura (GOODRICH; TAMASSIA, 2013).

Graças ao mecanismo de interrupção precoce, a complexidade temporal do algoritmo no melhor caso, caracterizado por uma entrada pré-ordenada, é reduzida para $\Theta(n)$, pois o algoritmo realiza apenas $n - 1$ comparações em uma única passagem pelo vetor (CORMEN et al., 2012; SEDGEWICK; WAYNE, 2011). No entanto, nos casos médio e pior (entradas aleatórias ou em ordem inversa), a complexidade assintótica permanece $\Theta(n^2)$, pois o sinalizador permanece ativado até as iterações finais do processo (CORMEN et al., 2012; SKIENA, 2020). A complexidade espacial mantém-se em O(1) (GOODRICH; TAMASSIA, 2013).

A principal vantagem desta abordagem em relação à versão clássica reside na sua capacidade adaptativa frente ao estado de ordenação da entrada, embora a manutenção da complexidade quadrática limite severamente sua viabilidade em cenários de alta demanda computacional (ASTRACHAN, 2003; LEVITIN, 2012).

2.3 Merge Sort

O *Merge Sort* é um algoritmo de ordenação de alto desempenho desenvolvido por John von Neumann, fundamentado rigorosamente no paradigma de Divisão e Conquista (CORMEN et al., 2012; KNUTH, 1998). O seu funcionamento desdobra-se em três fases encadeadas: a divisão recursiva do vetor de entrada em duas sublistas de dimensões equivalentes até se alcançarem estruturas unitárias; a conquista trivial das sublistas; e a intercalação (*merge*) ordenada e combinada dessas sublistas adjacentes para a reconstituição final do vetor completamente ordenado (SEDGEWICK; WAYNE, 2011; LEVITIN, 2012).

A complexidade temporal do *Merge Sort* é determinística e assintoticamente uniforme em $\Theta(n \log n)$ para os cenários de melhor, médio e pior caso (CORMEN et al., 2012). Essa garantia decorre da topologia da árvore de recursão, cuja profundidade é limitada a $\log_2 n$ níveis, associada ao custo linear $O(n)$ demandado pelo procedimento de intercalação em cada nível de processamento (KNUTH, 1998; SKIENA, 2020). Por outro lado, sua complexidade espacial é de $O(n)$, exigindo a alocação de um vetor temporário de suporte proporcional ao tamanho do conjunto de dados para executar a fusão dos elementos (SEDGEWICK; WAYNE, 2011).

As vantagens preponderantes do algoritmo abrangem a garantia previsível de desempenho temporal — imune a arranjos desfavoráveis da entrada — e a preservação da estabilidade (GOODRICH; TAMASSIA, 2013). Como limitações ressaltam-se o consumo adicional de memória primária $O(n)$ e o *overhead* computacional associado à pilha de chamadas recursivas e alocações dinâmicas para pequenos vetores (CORMEN et al., 2012; SKIENA, 2020).

3. Trabalhos Relacionados

Kaur e Garg (2018) analisaram a sensibilidade dos algoritmos de ordenação em função do estado inicial dos dados de entrada. Os autores mediram a quantidade de comparações e o tempo de processamento em cenários de ordenação aleatória, prévia e invertida para o *Bubble Sort* e algoritmos de divisão e conquista, concluindo que o *Bubble Sort* sem otimização mantém um comportamento puramente quadrático em todos os arranjos, enquanto o *Merge Sort* preserva a estabilidade temporal em

$\Theta(n \log n)$. A relação com este trabalho reside na fundamentação metodológica para avaliar a sensibilidade da entrada, diferenciando-se por incorporar explicitamente a variante do *Bubble Sort* com parada antecipada para mensurar o impacto direto da otimização adaptativa perante o *Merge Sort.*

Chowdhury et al. (2021) investigaram o impacto do mecanismo de interrupção antecipada via sinalizador booleano (*flag*) no algoritmo *Bubble Sort*. Por meio de experimentos computacionais com vetores em diferentes graus de ordenação, os autores demonstraram que a interrupção precoce reduz o tempo de execução no melhor caso para $\Omega(n)$, porém falha em mitigar a complexidade em cenários com alto índice de desordem. A correlação com esta pesquisa está na análise das limitações práticas do sinalizador de parada, avançando ao confrontar experimentalmente esse ganho no melhor caso com o custo computacional e o consumo de memória exigidos pelo *Merge Sort* sob grandes massas de dados.

Singh et al. (2023) realizaram um estudo comparativo entre algoritmos de ordenação de complexidade quadrática $O(n^2)$ e logarítmica $O(n \log n)$, focando na eficiência temporal e no consumo de memória. Os experimentos mediram o tempo de execução sobre vetores de 1.000 a 100.000 elementos gerados aleatoriamente, ratificando a superioridade dos algoritmos baseados em divisão e conquista sobre os métodos de varredura simples. A relação com esta pesquisa consiste no embasamento sobre a degradação temporal em função da escala da entrada, distinguindo-se ao incluir a versão otimizada do *Bubble Sort* e isolar os cenários de melhor, médio e pior caso para identificar o ponto exato de inflexão em que a otimização perde viabilidade prática.

Aditya et al. (2025) avaliaram o desempenho e a pegada espacial de algoritmos clássicos de ordenação, incluindo *Bubble Sort* e *Merge Sort*, em sistemas de processamento de dados. Os autores constataram que, embora o *Merge Sort* apresente tempo de processamento significativamente inferior em grandes escalas, ele impõe um *overhead* espacial de $O(n)$ devido à necessidade de vetores auxiliares na etapa de intercalação, contrastando com o caráter *in-place* $O(1)$ do *Bubble Sort*. O presente trabalho alinha-se a essa investigação ao considerar o *trade-off* entre tempo e espaço, diferenciando-se ao testar a resposta da flag de parada antecipada em entradas totalmente invertidas e previamente ordenadas para mapear a eficiência de ambas as abordagens.

4. Metodologia

Os algoritmos foram implementados na linguagem C e compilados com o Clang do LLVM-MinGW (UCRT), versão 23.1.4, com as opções -x c e -O2. Os experimentos foram realizados em um computador com processador Intel Core i5-7400 de 3,0 GHz, 16 GB de memória RAM e sistema operacional Windows 11. Foram avaliadas três implementações: *Bubble Sort* clássico, *Bubble Sort* otimizado (parada antecipada) e *Merge Sort* recursivo. No *Merge Sort*, o vetor auxiliar é alocado uma única vez a cada execução e reutilizado em todas as chamadas recursivas.

Para os testes, foram utilizados vetores com 1.000, 5.000, 10.000 e 50.000 elementos, em ordem aleatória, crescente e decrescente. Os vetores aleatórios foram gerados com a função rand() da biblioteca padrão de C, inicializada com srand(42).

Cada combinação de algoritmo, tamanho e tipo de entrada foi executada 10 vezes, utilizando os mesmos dados para as três implementações. O tempo de execução foi medido com a função QueryPerformanceCounter da API do Windows, convertendo-se a contagem do contador para milissegundos a partir de sua frequência. A média das execuções foi utilizada para comparar o desempenho dos algoritmos. Os resultados foram organizados em tabelas para análise posterior.

5. Resultados e Discussão

Tabela 1. Tempo médio de execução em milissegundos (ms) dos algoritmos em vetor aleatório.

Elementos	Bubble Sort	Bubble Otimizado	Merge Sort
1.000	1,064	0,494	0,045
5.000	28,344	14,315	0,253
10.000	120,818	88,902	0,521
50.000	3.047,870	3.036,914	2,973

Fonte: Elaborada pelo autor.

Na entrada aleatória, o *Bubble Sort* otimizado apresentou tempos inferiores aos da versão clássica em 1.000 elementos (redução de 53,6%), 5.000 elementos (49,5%) e 10.000 elementos (26,4%), e tempos praticamente iguais em 50.000 elementos (3.036,914 ms contra 3.047,870 ms, diferença de 0,4%). Essas reduções não podem ser atribuídas à parada antecipada. Em uma permutação aleatória, o número de passagens necessárias é muito próximo de $n - 1$ (da ordem de $n - \sqrt{(\pi n/2)}$), e o sinalizador só interrompe a execução nas últimas passagens, que são as mais curtas. Para 1.000 elementos, isso elimina bem menos de 1% das comparações. A convergência dos tempos em 50.000 elementos é coerente com esse comportamento esperado. Este trabalho não identificou a causa das reduções observadas nos tamanhos menores. Entre as explicações possíveis, não testadas, estão diferenças na geração de código do compilador para os dois laços e variações de medição, já que são reportadas apenas médias de 10 execuções. Essa incerteza é reforçada pelo crescimento irregular do tempo do *Bubble Sort* otimizado: de 5.000 para 10.000 elementos, o tempo aumentou 6,2 vezes, acima das 4 vezes esperadas para crescimento quadrático e das 4,3 vezes observadas na versão clássica. De 10.000 para 50.000 elementos (fator de 5 vezes na entrada), o tempo aumentou 25,2 vezes no *Bubble Sort* clássico, 34,2 vezes no otimizado e 5,7 vezes no *Merge Sort*, este último compatível com $\Theta(n \log n)$. Para 50.000 elementos, o *Merge Sort* (2,973 ms) foi aproximadamente 1.025 vezes mais rápido que o *Bubble Sort* clássico e 1.021 vezes mais rápido que o otimizado. Assim, apenas a equivalência entre as versões do *Bubble Sort* em 50.000 elementos é interpretada como resultado, e as diferenças em escalas menores são tratadas como inconclusivas.

Tabela 2. Tempo médio de execução em milissegundos dos algoritmos em vetor crescente.

Elementos	Bubble Sort	Bubble Otimizado	Merge Sort
1.000	0,222	0,001	0,031
5.000	5,562	0,003	0,143
10.000	21,934	0,006	0,318
50.000	565,047	0,030	1,670

Fonte: Elaborada pelo autor.

Com o vetor pré-ordenado (melhor caso), o *Bubble Sort* otimizado obteve o resultado mais expressivo do experimento. Como a primeira passagem não realiza nenhuma troca, o sinalizador interrompe a execução imediatamente, resultando em comportamento linear $\Theta(n)$ e permitindo ordenar 50.000 elementos em 0,030 ms. Nesse cenário, a versão otimizada foi cerca de 55,7 vezes mais rápida que o *Merge Sort* (1,670 ms), que mantém a divisão recursiva e a intercalação $\Theta(n \log n)$ independentemente do arranjo inicial. O *Bubble Sort* clássico, sem mecanismo de parada, executou todas as $n - 1$ passagens e as $n(n-1)/2$ comparações, mantendo o crescimento quadrático (565,047 ms para 50.000 elementos, com aumento de 25,8 vezes entre 10.000 e 50.000 elementos). Seu tempo foi menor que na entrada aleatória (3.047,870 ms), o que é coerente com a ausência de trocas de elementos nessa entrada.

Tabela 3. Tempo médio de execução em milissegundos dos algoritmos em vetor decrescente.

Elementos	Bubble Sort	Bubble Otimizado	Merge Sort
1.000	0,311	0,342	0,032
5.000	8,461	8,609	0,161
10.000	31,306	34,990	0,313
50.000	822,606	915,329	1,711

Fonte: Elaborada pelo autor.

Na entrada decrescente (pior caso), o sinalizador não interrompeu a execução antes da última passagem, pois todas as passagens realizaram trocas. A versão otimizada foi mais lenta que a clássica em todos os tamanhos: 10,0% (1.000 elementos), 1,7% (5.000), 11,8% (10.000) e 11,3% (50.000, com 915,329 ms contra 822,606 ms). Esse custo adicional é compatível com a atualização do sinalizador a cada troca. Como a diferença varia entre os tamanhos e não foi calculado desvio padrão, ela não é interpretada como uma sobrecarga constante. Ambas as versões apresentaram crescimento quadrático, com aumento de cerca de 26 vezes entre 10.000 e 50.000 elementos. O *Merge Sort* processou 50.000 elementos em 1,711 ms, sendo entre 480 e 535 vezes mais rápido que as versões do *Bubble Sort*. Os tempos das versões

quadráticas na entrada decrescente foram menores que na aleatória (822,606 ms e 915,329 ms contra 3.047,870 ms e 3.036,914 ms). Uma hipótese para esse resultado é a maior previsibilidade dos desvios (branch prediction), já que a condição de troca é verdadeira em todas as comparações do vetor inverso. Essa hipótese não foi verificada neste trabalho.

6. Considerações Finais

Este trabalho apresentou uma avaliação experimental comparativa entre o *Bubble Sort* clássico, sua versão otimizada por parada antecipada e o *Merge Sort*, analisando o impacto da escala e da ordenação inicial dos dados no tempo de execução.

Os resultados confirmaram a superioridade do *Merge Sort* nos cenários aleatório e decrescente, com 2,973 ms e 1,711 ms para 50.000 elementos, comportamento coerente com a complexidade $\Theta(n \log n)$.

A otimização com sinalizador no *Bubble Sort* mostrou eficácia restrita à entrada previamente ordenada. No vetor crescente, a interrupção precoce permitiu ordenar 50.000 elementos em 0,030 ms, cerca de 55,7 vezes mais rápido que o *Merge Sort*. Na entrada aleatória com 50.000 elementos, as duas versões do *Bubble Sort* tiveram tempos equivalentes (diferença de 0,4%), e na entrada decrescente a versão otimizada foi mais lenta que a clássica em todos os tamanhos, em até 11,8%. As reduções de tempo da versão otimizada na entrada aleatória com 1.000 a 10.000 elementos não são explicadas pela parada antecipada e permanecem inconclusivas. Por não possuir critério de parada, o *Bubble Sort* clássico realiza $n(n-1)/2$ comparações em qualquer arranjo, e seus tempos variaram entre os arranjos (para 50.000 elementos: 3.047,870 ms no aleatório, 565,047 ms no crescente e 822,606 ms no decrescente).

As conclusões limitam-se à escala máxima de 50.000 elementos, ao ambiente computacional utilizado e à medição por média de 10 execuções, sem cálculo de desvio padrão. Não foram medidos o consumo de memória, o tempo de leitura de dados nem eventos de arquitetura, como cache misses e falhas de predição de desvio. Para trabalhos futuros, recomenda-se repetir os experimentos com mais execuções e desvio padrão, instrumentar a contagem de comparações e trocas, utilizar *profilers* de hardware para investigar as diferenças observadas na Tabela 1 e estender a análise comparativa para algoritmos adaptativos híbridos como o *Timsort*.

Declaração de Uso de IA

O Claude Sonnet 5.5 foi utilizado exclusivamente para revisão ortográfica e semântica do texto. Todas as análises, dados e conclusões apresentadas permanecem sob responsabilidade dos autores.

Referências

ADITYA, R. et al. Performance Analysis of Sorting Algorithm in Student Data Processing. **Journal of Enhanced Studies in Informatics and Computer Applications**, v. 4, n. 1, p. 54-61, 2025.

ASTRACHAN, O. Bubble sort: an archaeological algorithmic analysis. **ACM SIGCSE Bulletin**, v. 35, n. 1, p. 1-5, 2003.

CHOWDHURY, M. H. et al. Empirical Analysis of Early Termination Optimizations in Elementary Sorting Algorithms. **International Journal of Computer Applications**, v. 183, n. 12, p. 18-24, 2021.

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, M. T.; TAMASSIA, R. **Estrutura de dados e algoritmos em Java**. 5. ed. Porto Alegre: Bookman, 2013.

KAUR, A.; GARG, R. Experimental Performance Analysis of Comparison-Based Sorting Algorithms. **International Journal of Computer Sciences and Engineering**, v. 6, n. 5, p. 312-317, 2018.

KNUTH, D. E. **The Art of Computer Programming, Volume 3: Sorting and Searching**. 2. ed. Upper Saddle River: Addison-Wesley, 1998.

LEVITIN, A. **Introduction to the Design and Analysis of Algorithms**. 3. ed. Boston: Pearson, 2012.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.

SINGH, R. et al. A Comparative Study of Sorting Algorithms: Efficiency and Performance. **International Journal of Advanced Computer Science and Applications**, v. 14, n. 2, p. 202-209, 2023.

SKIENA, S. S. **The Algorithm Design Manual**. 3. ed. Cham: Springer, 2020.